

lab_ensemble_v_2026

June 28, 2026

1 Laboratorium: Metody Zespołowe (Ensemble Learning)

Metody Sztucznej Inteligencji | Politechnika Gdańska | sem. letni 2025/2026

Imię i nazwisko:	Igor Józefowicz
Nr indeksu:	193257
Data wykonania:	05.05.2026 r.

1.0.1 Cel laboratorium

Praktyczne poznanie metod zespołowych: **Random Forest**, **AdaBoost** i **Gradient Boosting**.

1.0.2 Organizacja zajęć (90 min)

Zadanie	Temat	Czas
Zadanie 1	Random Forest – bootstrap, OOB, feature importance (zbiór syntetyczny)	~30 min
Zadanie 2	AdaBoost – wrażliwość na szum, granice decyzyjne (make_moons)	~30 min
Zadanie 3	Gradient Boosting – pseudoreszty, tuning i porównanie metod	~30 min

1.0.3 Zasady

- W komórkach oznaczonych **Twój kod** uzupełnij brakujące fragmenty (oznaczone # TODO).
- Eksperymentuj z parametrami tam, gdzie jest to sugerowane – **zmieniaj wartości i obserwuj efekty**.
- Odpowiedzi na pytania wpisuj w komórkach **Tvoja odpowiedź**.
- Do oceny liczy się zarówno **działający kod**, jak i **jakość wniosków**.

1.1 Konfiguracja środowiska

Uruchom poniższe komórki **raz** na początku zajęć.

```
[1]: %pip install -q scikit-learn matplotlib numpy pandas seaborn
```

Note: you may need to restart the kernel to use updated packages.

```
WARNING: Ignoring invalid distribution ~atplotlib  
(c:\Users\jozef\anaconda3\Lib\site-packages)
```

```
WARNING: Ignoring invalid distribution ~atplotlib  
(c:\Users\jozef\anaconda3\Lib\site-packages)
```

```
WARNING: Ignoring invalid distribution ~atplotlib  
(c:\Users\jozef\anaconda3\Lib\site-packages)
```

```
[2]: import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
from sklearn.datasets import make_classification, make_moons, make_regression  
from sklearn.model_selection import train_test_split, cross_val_score,  
    ↳RandomizedSearchCV  
from sklearn.ensemble import (RandomForestClassifier, AdaBoostClassifier,  
    ↳GradientBoostingClassifier,  
    ↳GradientBoostingRegressor)  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.metrics import accuracy_score, f1_score, mean_squared_error  
from sklearn.inspection import permutation_importance  
  
import warnings; warnings.filterwarnings('ignore')  
sns.set_theme(style='whitegrid')  
plt.rcParams.update({'figure.dpi': 120, 'font.size': 11})  
RANDOM_STATE = 42  
print("Biblioteki załadowane")
```

Biblioteki załadowane

1.2 Zadanie 1 – Random Forest krok po kroku (30 min)

1.2.1 Czym jest Random Forest?

Random Forest to metoda **bagging** – trenuje wiele drzew decyzyjnych **niezależnie** na próbach bootstrapowych, a następnie agreguje ich predykcje przez głosowanie (klasyfikacja) lub uśrednianie (regresja).

Kluczowa idea: Dekorelacja drzew przez losowy dobór cech (*random subspace*) obniża **wariancję** przy zachowaniu niskiego odchylenia. Dlatego RF nie przeuczy się przy rosnącej liczbie drzew (Breiman 2001).

1.2.2 Zbiór danych – syntetyczny `make_classification`

Używamy **syntetycznego zbioru** z 2000 próbkami i **50 cechami** (tylko 5 informatywnych, 20 redundantnych, 10 powtórzonych, 5% szumu etykiet).

Dzięki temu efekt liczby drzew i rola `max_features` są **wyraźnie widoczne** (różnica OOB rzędu 5–8 pp), co byłoby niewidoczne na prostszych zbiorach.

1.2.3 Plan:

1. Załaduj dane i podziel na zbiór treningowy/testowy
2. Zbuduj Random Forest z OOB score
3. Zbadaj wpływ liczby drzew na wynik
4. Przeanalizuj ważność cech (MDI vs Permutation Importance)
5. Mini-eksperyment: `max_features='sqrt'` vs `max_features=None`

```
[3]: # === KROK 1: Załaduj dane ===  
# Parametry:  
#   n_features=50    - łączna liczba cech  
#   n_informative=5  - tylko 5 cech niesie informację  
#   n_redundant=20   - kombinacje liniowe cech informatywnych  
#   n_repeated=10    - kopie istniejących cech  
#   flip_y=0.05      - 5% etykiet losowo odwrócone (szum)  
  
X, y = make_classification(  
    n_samples=2000, n_features=50, n_informative=5,  
    n_redundant=20, n_repeated=10, flip_y=0.05,  
    random_state=RANDOM_STATE  
)  
feature_names = np.array([f'cecha_{i}' for i in range(X.shape[1])])  
  
print(f"Kształt X: {X.shape}")  
print(f"Liczba klas: {len(np.unique(y))}")  
print(f"Udział klasy 1: {y.mean():.3f}")
```

Kształt X: (2000, 50)

Liczba klas: 2

Udział klasy 1: 0.500

```
[5]: # === KROK 2: Podział danych ===  
# TODO: Podziel X i y na zbiór treningowy (75%) i testowy (25%)  
#       Użyj stratify=y i random_state=RANDOM_STATE  
  
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y,  
    ↪ random_state=RANDOM_STATE, test_size=0.25)  
  
print(f"X_train: {X_train.shape}, X_test: {X_test.shape}")  
print(f"Udział klasy 1 w train: {y_train.mean():.3f}, w test: {y_test.mean():.  
    ↪ 3f}")
```

X_train: (1500, 50), X_test: (500, 50)
Udział klasy 1 w train: 0.501, w test: 0.500

1.2.4 Krok 3: Budowa Random Forest z OOB

Podczas bootstrap sampling ~36.8% próbek nie trafia do danego drzewa – to próbki **Out-of-Bag (OOB)**. Ustawiając `oob_score=True`, RF automatycznie oblicza dokładność na próbkach OOB – to **darmowy estymator błędu generalizacji** bez potrzeby walidacji krzyżowej.

Matematycznie: frakcja unikalnych próbek w próbie bootstrapowej:

$$1 - \lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right)^n = 1 - \frac{1}{e} \approx 63,2\%$$

Pozostałe $\approx 36,8\%$ to OOB.

Zadanie: Utwórz `RandomForestClassifier` z: - `n_estimators=100` - `max_features='sqrt'` ($\sqrt{p}$ cech w każdym węźle) - `oob_score=True`

```
[6]: # === KROK 3: Random Forest z OOB ===
# TODO: Utwórz i wytrenuj RandomForestClassifier z n_estimators=100,
#       max_features='sqrt', oob_score=True, random_state=RANDOM_STATE

rf = RandomForestClassifier(max_features='sqrt', oob_score=True,
    ↪random_state=RANDOM_STATE)
rf.fit(X_train, y_train)

rf_pred = rf.predict(X_test)
print(f"Dokładność na zbiorze testowym: {accuracy_score(y_test, rf_pred):.4f}")
print(f"F1 na zbiorze testowym: {f1_score(y_test, rf_pred):.4f}")
print(f"OOB score (darmowa walidacja): {rf.oob_score_: .4f}")
```

Dokładność na zbiorze testowym: 0.8880
F1 na zbiorze testowym: 0.8915
OOB score (darmowa walidacja): 0.9107

1.2.5 Krok 4: Wpływ liczby drzew na wynik

Na wykładzie podkreślono, że RF **nie przeuczy się** przy rosnącej liczbie drzew – błąd generalizacji zbiega asymptotycznie (Breiman 2001). Na zbiorze syntetycznym zbieżność jest wyraźna.

Zadanie: Dla każdej wartości `n_estimators` z listy wytrenuj RF, zapisz F1 testowe i OOB score.

```
[7]: # === KROK 4: Wpływ liczby drzew ===
n_estimators_grid = [5, 10, 25, 50, 100, 150, 200, 300]
results = []

for n in n_estimators_grid:
    # TODO: Utwórz RF z n drzewami, max_features='sqrt', oob_score=True
    rf_n = RandomForestClassifier(n_estimators=n, max_features='sqrt',
    ↪oob_score=True)
    rf_n.fit(X_train, y_train)
```

```

rf_pred_n = rf_n.predict(X_test)
results.append({
    'n_estimators': n,
    'f1_test':      f1_score(y_test, rf_pred_n),
    'oob_score':    rf_n.oob_score_,
    'acc_test':     accuracy_score(y_test, rf_pred_n),
})

results_df = pd.DataFrame(results)
print(results_df.to_string(index=False))

```

n_estimators	f1_test	oob_score	acc_test
5	0.866667	0.811333	0.864
10	0.866935	0.860667	0.868
25	0.875969	0.900000	0.872
50	0.891473	0.903333	0.888
100	0.891051	0.898000	0.888
150	0.888454	0.899333	0.886
200	0.883721	0.906667	0.880
300	0.891013	0.903333	0.886

```

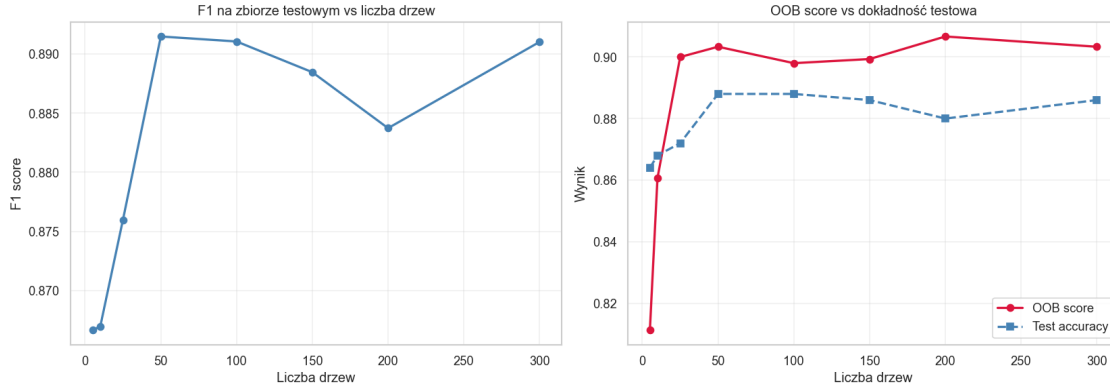
[8]: # Wizualizacja - wpływ liczby drzew
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(results_df['n_estimators'], results_df['f1_test'],
              'o-', color='steelblue', lw=2)
axes[0].set_xlabel('Liczba drzew')
axes[0].set_ylabel('F1 score')
axes[0].set_title('F1 na zbiorze testowym vs liczba drzew')
axes[0].grid(True, alpha=0.3)

axes[1].plot(results_df['n_estimators'], results_df['oob_score'],
              'o-', color='crimson', lw=2, label='OOB score')
axes[1].plot(results_df['n_estimators'], results_df['acc_test'],
              's--', color='steelblue', lw=2, label='Test accuracy')
axes[1].set_xlabel('Liczba drzew')
axes[1].set_ylabel('Wynik')
axes[1].set_title('OOB score vs dokładność testowa')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



1.2.6 Krok 5: Ważność cech – MDI vs Permutation Importance

Na wykładzie omówiono dwie miary ważności cech: - **MDI (Mean Decrease Impurity)**: suma poprawy Gini po wszystkich podziałach używających danej cechy. **Uwaga:** MDI preferuje cechy o wielu unikalnych wartościach i może zawyżać rangę cech redundantnych. - **Permutation Importance**: mierzy spadek jakości modelu po losowym przemieszaniu wartości danej cechy. Jest bardziej wiarygodna.

Zadanie: Oblicz obie miary ważności i porównaj Top 10 cech.

```
[9]: # === KROK 5: MDI vs Permutation Importance ===

# MDI - wbudowana ważność z RF
mdi = pd.Series(rf.feature_importances_, index=feature_names).
    ↪sort_values(ascending=False)

# TODO: Oblicz Permutation Importance używając permutation_importance()
#       Użyj rf wytrenowanego na X_train/y_train, ewaluuj na X_test/y_test
#       n_repeats=10, random_state=RANDOM_STATE
perm = permutation_importance(estimator=rf, X=X_test, y=y_test, n_repeats=10,
    ↪random_state=RANDOM_STATE)
perm_imp = pd.Series(perm.importances_mean, index=feature_names).
    ↪sort_values(ascending=False)

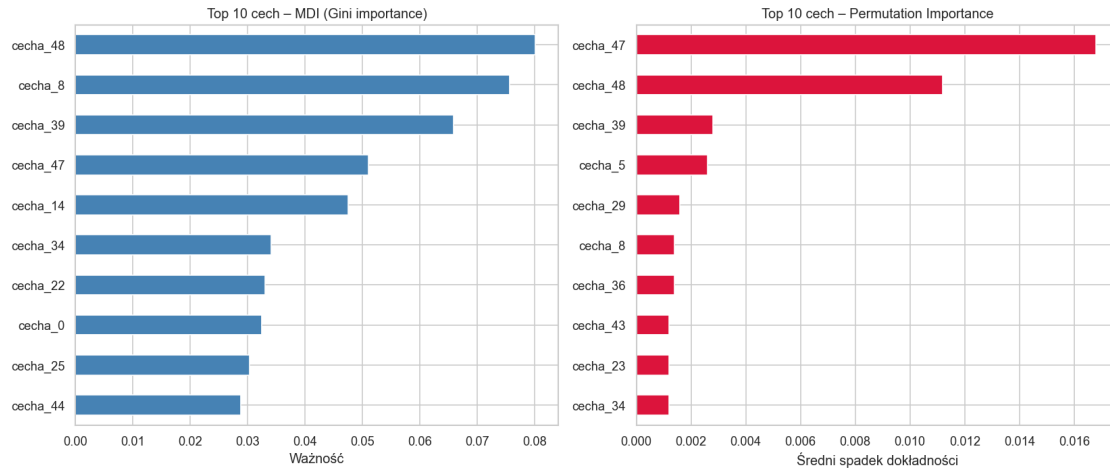
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

mdi.head(10).sort_values().plot(kind='barh', ax=axes[0], color='steelblue')
axes[0].set_title('Top 10 cech - MDI (Gini importance)')
axes[0].set_xlabel('Ważność')

perm_imp.head(10).sort_values().plot(kind='barh', ax=axes[1], color='crimson')
axes[1].set_title('Top 10 cech - Permutation Importance')
axes[1].set_xlabel('Średni spadek dokładności')
```

```
plt.tight_layout()
plt.show()

print("Top 5 cech wg MDI: ", list(mdi.head(5).index))
print("Top 5 cech wg Perm: ", list(perm_imp.head(5).index))
```



Top 5 cech wg MDI: ['cecha_48', 'cecha_8', 'cecha_39', 'cecha_47', 'cecha_14']
 Top 5 cech wg Perm: ['cecha_47', 'cecha_48', 'cecha_39', 'cecha_5', 'cecha_29']

1.2.7 Krok 6 (Mini-eksperyment): max_features='sqrt' vs max_features=None

Zadanie: Wytrenuj dwa modele RF – jeden z max_features='sqrt', drugi z max_features=None (wszystkie cechy). Porównaj OOB score i F1 testowe. Zastanów się, dlaczego losowy wybór cech jest kluczowy gdy tylko 5 z 50 cech niesie informację.

```
[10]: # === KROK 6: max_features - porównanie ===
for mf in ['sqrt', None]:
    # TODO: Utwórz i wytrenuj RF z n_estimators=100, oob_score=True
    #       i podanym max_features
    rf_mf = RandomForestClassifier(n_estimators=100, oob_score=True,
    ↪max_features=mf)
    rf_mf.fit(X_train, y_train)
    pred_mf = rf_mf.predict(X_test)
    print(f"max_features={str(mf):6s} | "
          f"OOB={rf_mf.oob_score_:.4f} | "
          f"F1={f1_score(y_test, pred_mf):.4f}")
```

```
max_features=sqrt      | OOB=0.9000 | F1=0.8940
max_features=None     | OOB=0.9013 | F1=0.8745
```

1.2.8 Pytania do Zadania 1

P1.1 Jak zmienia się F1 wraz ze wzrostem liczby drzew? Przy ilu drzewach wynik przestaje znacząco rosnąć?

P1.2 Jak OOB score koreluje z dokładnością testową? Czy można na nim polegać jako estymatorze błędu generalizacji?

P1.3 Porównaj Top 5 cech wg MDI i Permutation Importance. Czy rankingi się różnią? Która miara lepiej wyłania cechy naprawdę informatywne?

P1.4 Dlaczego z `max_features=None` wynik jest gorszy niż z `max_features='sqrt'`? Odwołaj się do pojęcia *dekorelacji drzew* z wykładu.

Twoja odpowiedź:

P1.1:

F1 dynamicznie wzrasta do około 50 drzew. Potem wraz ze wzrostem do około 200 drzew spada i następnie do około 300 drzew ponownie wzrasta. Wyniki dla poniżej 50 drzew są wyraźnie słabsze.

P1.2:

Jest zauważalna pewna korelacja. Dla liczby drzew < 50 widać też zauważalnie niższe test accuracy. Potem kiedy OOB score wzrasta, również test accuracy wzrasta. Przy liczbie drzew powyżej 50 obie wartości pozostają na podobnym poziomie.

P1.3:

W obu rankingach TOP 5 pojawiają się cechy numer: 48, 39, 47

Natomiast nie pojawiają się w obu rankingach cechy numer: 8 (z Gini importance), 14 (z Gini importance), 29 (z Permutation importance), 5 (z Permutation importance)

Widać zauważalną różnicę między tymi rankingami. Permutation importance zauważa 2 najważniejsze cechy -> 47 i 48. Reszta cech zauważalnie mocno odstaje.

Natomiast w MDI (Gini importance) te cechy są bardziej “zbalansowane” pod względem ważności. Tutaj nie widać aż tak drastycznych skoków jak w Permutation importance.

Cechy naprawdę informatywne lepiej powinien znajdować Permutation importance, ale jest to sposób wolniejszy, używany na zbiorze walidacyjnym lub testowym, ale oferujący mniejszy bias.

P1.4:

Z `max_features=None` wynik jest gorszy o około 2 punkty procentowe, ponieważ Bagging redukuje wariancję przez dekorelację drzew zachowując jednocześnie nieskie odchylenie.

1.3 Zadanie 2 – AdaBoost i wpływ szumu (30 min)

1.3.1 Czym jest AdaBoost?

AdaBoost (Adaptive Boosting) (Freund & Schapire, 1995) to metoda **boosting** – trenuje modele **sekwencyjnie**, gdzie każdy kolejny klasyfikator skupia się na próbkach, które poprzedni źle sklasyfikował.

Mechanizm (z wykładu): - Inicjalizacja wag: $w_i^{(1)} = \frac{1}{n}$ - Błąd ważony: $\epsilon_t = \sum_{i: h_t(x_i) \neq y_i} w_i^{(t)}$
- Waga klasyfikatora: $\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$ - Aktualizacja wag: $w_i^{(t+1)} \propto w_i^{(t)} \exp(-\alpha_t y_i h_t(x_i))$ -
Predykcja: $H(x) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right)$

Wada: Wrażliwość na **szum i outliery** – próbki z błędnymi etykietami dostają coraz większe wagi.

1.3.2 Plan:

1. Przygotuj dane `make_moons`
2. Porównaj degradację jakości AdaBoost i RF przy rosnącym szumie etykiet
3. Zwizualizuj granice decyzyjne przy różnych poziomach szumu
4. Zbadaj wpływ liczby estymatorów na F1

```
[11]: # === KROK 1: Dane make_moons ===
X_moons, y_moons = make_moons(n_samples=1000, noise=0.25,
    ↪random_state=RANDOM_STATE)

X_train_moons, X_test_moons, y_train_moons, y_test_moons = train_test_split(
    X_moons, y_moons, test_size=0.25, stratify=y_moons,
    ↪random_state=RANDOM_STATE
)
print(f"Kształt X_moons: {X_moons.shape}, Udział klasy 1: {y_moons.mean():.3f}")
```

Kształt X_moons: (1000, 2), Udział klasy 1: 0.500

```
[10]: # === KROK 2: Eksperyment z szumem ===
def add_label_noise(y, frac, random_state=42):
    """Odwraca losowo `frac` frakcję etykiet."""
    rng = np.random.RandomState(random_state)
    y_noisy = np.array(y, dtype=int)
    n_flip = int(frac * len(y_noisy))
    idx = rng.choice(len(y_noisy), size=n_flip, replace=False)
    y_noisy[idx] = 1 - y_noisy[idx]
    return y_noisy

noise_levels = [0.0, 0.05, 0.10, 0.20, 0.30, 0.40]
ada_f1 = []
rf_f1 = []

for noise in noise_levels:
```

```

y_train_noisy = add_label_noise(y_train_moons, frac=noise)

# TODO: Utwórz AdaBoostClassifier z DecisionTreeClassifier(max_depth=1)
#       jako base estimator, n_estimators=100, learning_rate=1.0
ada_n = AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1),
↪n_estimators=100, learning_rate=1.0)
ada_n.fit(X_train_moons, y_train_noisy)
ada_f1.append(f1_score(y_test_moons, ada_n.predict(X_test_moons)))

# TODO: Utwórz RandomForestClassifier z n_estimators=100,
↪max_features='sqrt'
rf_n = RandomForestClassifier(n_estimators=100, max_features='sqrt')
rf_n.fit(X_train_moons, y_train_noisy)
rf_f1.append(f1_score(y_test_moons, rf_n.predict(X_test_moons)))

print(f"{'Szum':>6} | {'AdaBoost F1':>12} | {'RF F1':>8}")
for nv, af, rfv in zip(noise_levels, ada_f1, rf_f1):
    print(f"{nv:.2f} | {af:.4f} | {rfv:.4f}")

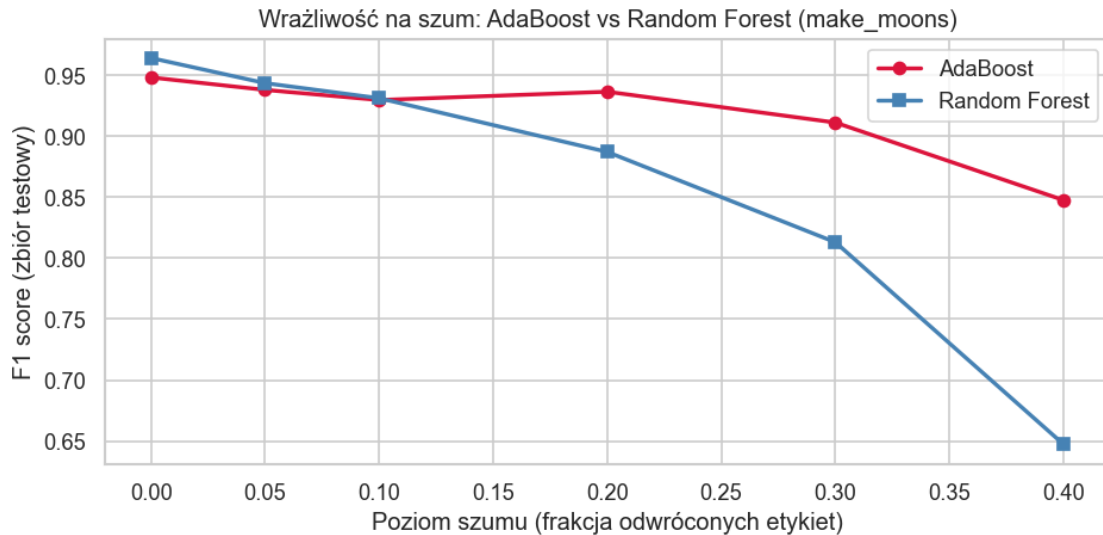
```

Szum	AdaBoost F1	RF F1
0.00	0.9482	0.9641
0.05	0.9380	0.9435
0.10	0.9297	0.9312
0.20	0.9365	0.8871
0.30	0.9112	0.8130
0.40	0.8476	0.6473

```

[11]: # === KROK 3: Wykres F1 vs szum ===
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(noise_levels, ada_f1, 'o-', label='AdaBoost', color='crimson', lw=2)
ax.plot(noise_levels, rf_f1, 's-', label='Random Forest', color='steelblue',
↪lw=2)
ax.set_xlabel('Poziom szumu (frakcja odwróconych etykiet)')
ax.set_ylabel('F1 score (zbiór testowy)')
ax.set_title('Wrażliwość na szum: AdaBoost vs Random Forest (make_moons)')
ax.legend()
plt.tight_layout()
plt.show()

```



1.3.3 Krok 4: Granice decyzyjne przy różnych poziomach szumu

Zadanie: Zwizualizuj granice decyzyjne AdaBoost i RF dla szumu 0% i 30%. Obserwuj, jak AdaBoost reaguje na zaszumione próbki.

```
[15]: # === KROK 4: Granice decyzyjne ===
def plot_decision_boundary(ax, model, X, y, title):
    h = 0.02
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                          np.arange(y_min, y_max, h))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    ax.contourf(xx, yy, Z, alpha=0.3, cmap='RdBu')
    ax.scatter(X[:, 0], X[:, 1], c=y, cmap='RdBu',
               edgecolors='k', s=20, alpha=0.7)
    ax.set_title(title)

fig, axes = plt.subplots(2, 2, figsize=(12, 9))

for col, noise_val in enumerate([0.0, 0.30]):
    y_train_noisy = add_label_noise(y_train_moons, frac=noise_val)

    ada_vis = AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=1),
        n_estimators=100, learning_rate=1.0, random_state=RANDOM_STATE)
    ada_vis.fit(X_train_moons, y_train_noisy)

    rf_vis = RandomForestClassifier(
```

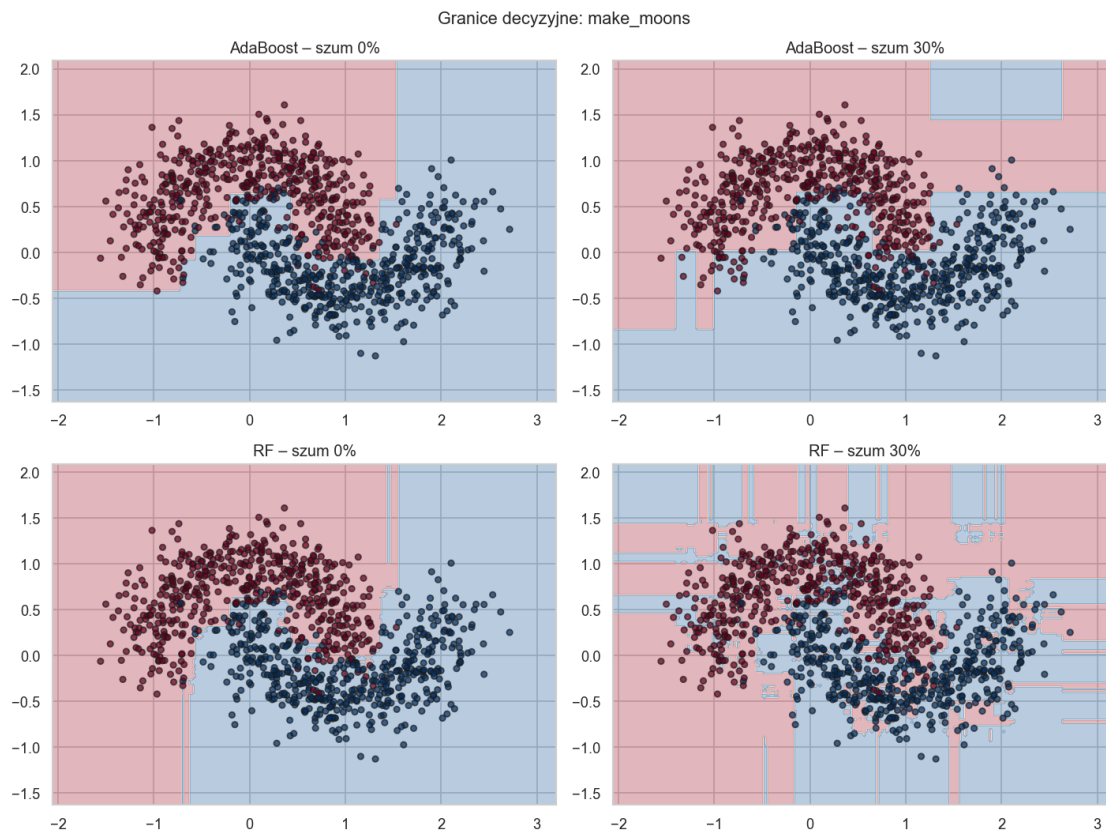
```

n_estimators=100, max_features='sqrt', random_state=RANDOM_STATE)
rf_vis.fit(X_train_moons, y_train_noisy)

plot_decision_boundary(axes[0, col], ada_vis, X_moons, y_moons,
                        f'AdaBoost - szum {int(noise_val*100)}%')
plot_decision_boundary(axes[1, col], rf_vis, X_moons, y_moons,
                        f'RF - szum {int(noise_val*100)}%')

plt.suptitle('Granice decyzyjne: make_moons', fontsize=13)
plt.tight_layout()
plt.show()

```



1.3.4 Krok 5: Wpływ liczby estymatorów w AdaBoost

Zadanie: Zbadaj, jak F1 AdaBoost zmienia się wraz z liczbą estymatorów. Czy AdaBoost może się przeczyć przy zbyt dużej liczbie estymatorów na zaszumionych danych?

```

[16]: # === KROK 5: Liczba estymatorów w AdaBoost (czyste vs zaszumione dane) ===
n_est_grid = [5, 10, 25, 50, 100, 200, 300]
ada_clean = []
ada_noisy = []

```

```

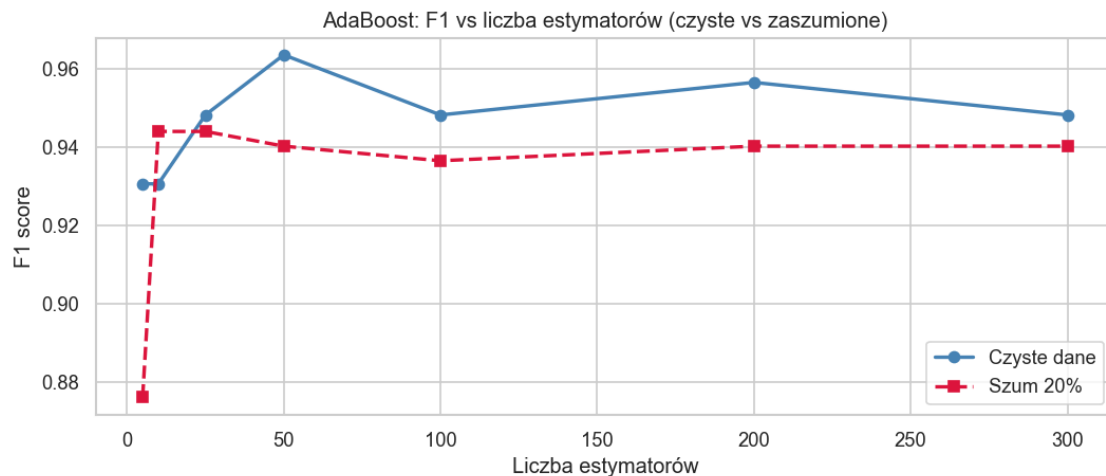
y_train_noisy30 = add_label_noise(y_train_moons, frac=0.20)

for n in n_est_grid:
    # TODO: Wytrenuj AdaBoost z n estymatorami na czystych danych
    ada_c = AdaBoostClassifier(n_estimators=n)
    ada_c.fit(X_train_moons, y_train_moons)
    ada_clean.append(f1_score(y_test_moons, ada_c.predict(X_test_moons)))

    # TODO: Wytrenuj AdaBoost z n estymatorami na danych z 20% szumem
    ada_n = AdaBoostClassifier(n_estimators=n)
    ada_n.fit(X_train_moons, y_train_noisy30)
    ada_noisy.append(f1_score(y_test_moons, ada_n.predict(X_test_moons)))

fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(n_est_grid, ada_clean, 'o-', label='Czyste dane',
        color='steelblue', lw=2)
ax.plot(n_est_grid, ada_noisy, 's--', label='Szum 20%',
        color='crimson', lw=2)
ax.set_xlabel('Liczba estymatorów')
ax.set_ylabel('F1 score')
ax.set_title('AdaBoost: F1 vs liczba estymatorów (czyste vs zaszumione)')
ax.legend()
plt.tight_layout()
plt.show()

```



1.3.5 Pytania do Zadania 2

P2.1 Opisz, jak F1 AdaBoost i RF zmienia się wraz ze wzrostem szumu. Która metoda jest bardziej odporna? Dlaczego mechanizm ważenia próbek AdaBoost jest problematyczny przy szumie?

P2.2 Opisz różnice w granicach decyzyjnych AdaBoost i RF przy szumie 30%. Która granica jest bardziej złożona i dlaczego?

P2.3 Czy AdaBoost przeuczy się przy dużej liczbie estymatorów na zaszumionych danych? Jak to się ma do zachowania RF (Zadanie 1, Krok 4)?

P2.4 Przypomnij z wykładu: AdaBoost minimalizuje stratę $L = \sum_i \exp(-y_i F(x_i))$. Dlaczego funkcja eksponencjalna jest szczególnie wrażliwa na outliery?

Twoja odpowiedź:

P2.1:

Zaobserwowano spadek jakości obu metod wraz ze wzrostem szumu, ale spadek nie był jednakowy. AdaBoost zmienił F1 z 0.9482 dla danych czystych do 0.8476 przy szumie 40%, natomiast RF spadł z 0.9641 do 0.6473. W tym eksperymencie AdaBoost okazał się odporniejszy przy dużym szumie, choć mechanizm ważenia próbek może być problematyczny, ponieważ błędnie oznaczone obserwacje dostają coraz większą wagę.

P2.2:

Przy szumie 30% granice RF są bardziej poszatkowane i lokalnie dopasowane do zaburzonych punktów. Granice AdaBoost wyglądają bardziej regularnie, chociaż również pojawiają się lokalne odchylenia wynikające z nacisku na trudne obserwacje.

P2.3:

Przy dużej liczbie estymatorów AdaBoost może wzmacniać wpływ zaszumionych etykiet, więc rośnie ryzyko dopasowania do błędnych przykładów. RF zwykle stabilizuje wynik przez uśrednianie wielu drzew i losowanie cech, dlatego po osiągnięciu wystarczającej liczby drzew poprawa jest niewielka.

P2.4:

Funkcja eksponencjalna silnie karze punkty klasyfikowane z dużym błędem. Pojedynczy outlier może więc uzyskać bardzo duży wpływ na kolejne iteracje, co zwiększa wrażliwość AdaBoost na obserwacje odstające i błędne etykiety.

1.4 Zadanie 3 – Gradient Boosting: pseudoreszty, tuning i porównanie (30 min)

1.4.1 Czym jest Gradient Boosting?

Gradient Boosting (Friedman, 2001) traktuje budowę modelu jako **optymalizację w przestrzeni funkcji**. Każde kolejne drzewo aproksymuje **ujemny gradient funkcji straty** – czyli pseudoreszty:

$$r_i^{(m)} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}$$

Dla MSE: $r_i = y_i - F_{m-1}(x_i)$ (prawdziwe reszty).

Dla log-loss: $r_i = y_i - p_i$.

Kluczowa różnica od AdaBoost: AdaBoost zmienia *wagi próbek*, Gradient Boosting zmienia *etykiety* (pseudoreszty).

Hiperparametry: - `n_estimators` – liczba drzew - `learning_rate` $\nu \in (0, 1]$ – małe ν = lepsza generalizacja, ale potrzeba więcej drzew - `max_depth` – głębokość drzew (typowo 3–8) - `subsample` < 1 – Stochastic GB, dodatkowa regularyzacja

1.4.2 Plan:

1. Demonstracja pseudoreszt na prostym przykładzie regresji
2. Tuning hiperparametrów GBM na zbiorze klasyfikacyjnym
3. Porównanie RF, AdaBoost i GBM

```
[19]: # === KROK 1: Demonstracja pseudoreszt (regresja) ===
# Tworzymy prosty zbiór regresyjny 1D
np.random.seed(RANDOM_STATE)
X_reg = np.sort(np.random.uniform(0, 6, 80))[:, np.newaxis]
y_reg = np.sin(X_reg).ravel() + np.random.normal(0, 0.3, 80)

# Trenujemy GBM krok po kroku i śledzimy pseudoreszty
from sklearn.tree import DecisionTreeRegressor

F = np.full_like(y_reg, y_reg.mean()) # F_0 = średnia
learning_rate = 0.5
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for m, ax in enumerate(axes):
    # TODO: Oblicz pseudoreszty (dla MSE:  $r = y - F$ )
    r = y_reg - F

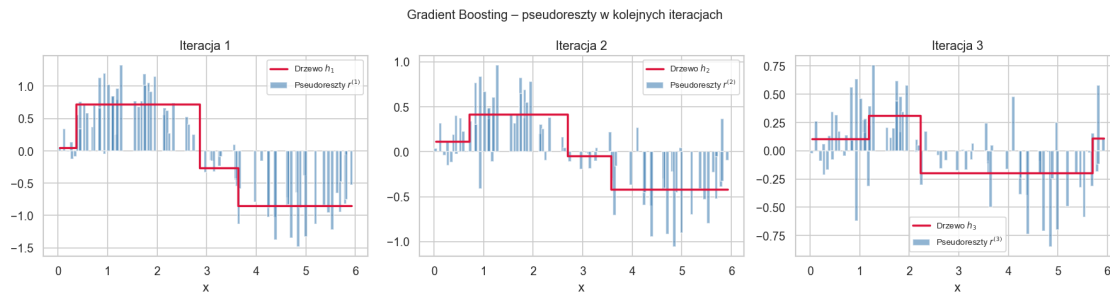
    # Dopasuj drzewo do pseudoreszt
    tree = DecisionTreeRegressor(max_depth=2, random_state=RANDOM_STATE)
    tree.fit(X_reg, r)
    h = tree.predict(X_reg)

    # Aktualizuj predykcję
    F = F + learning_rate * h

    ax.bar(X_reg.ravel(), r, width=0.06, alpha=0.6, color='steelblue',
           label=f'Pseudoreszty  $r^{\{(m+1)\}}$ ')
    ax.step(X_reg.ravel(), h, color='crimson', lw=2, where='mid',
           label=f'Drzewo  $h_{\{m+1\}}$ ')
    ax.set_title(f'Iteracja {m+1}')
    ax.set_xlabel('x')
    ax.legend(fontsize=8)

plt.suptitle('Gradient Boosting - pseudoreszty w kolejnych iteracjach',
             ↪fontsize=12)
```

```
plt.tight_layout()
plt.show()
```



1.4.3 Krok 2: Tuning GBM – learning rate vs liczba drzew

Na wykładzie podkreślono, że małe `learning_rate` (ν) daje lepszą generalizację kosztem potrzeby większej liczby drzew. Zbadajmy ten kompromis.

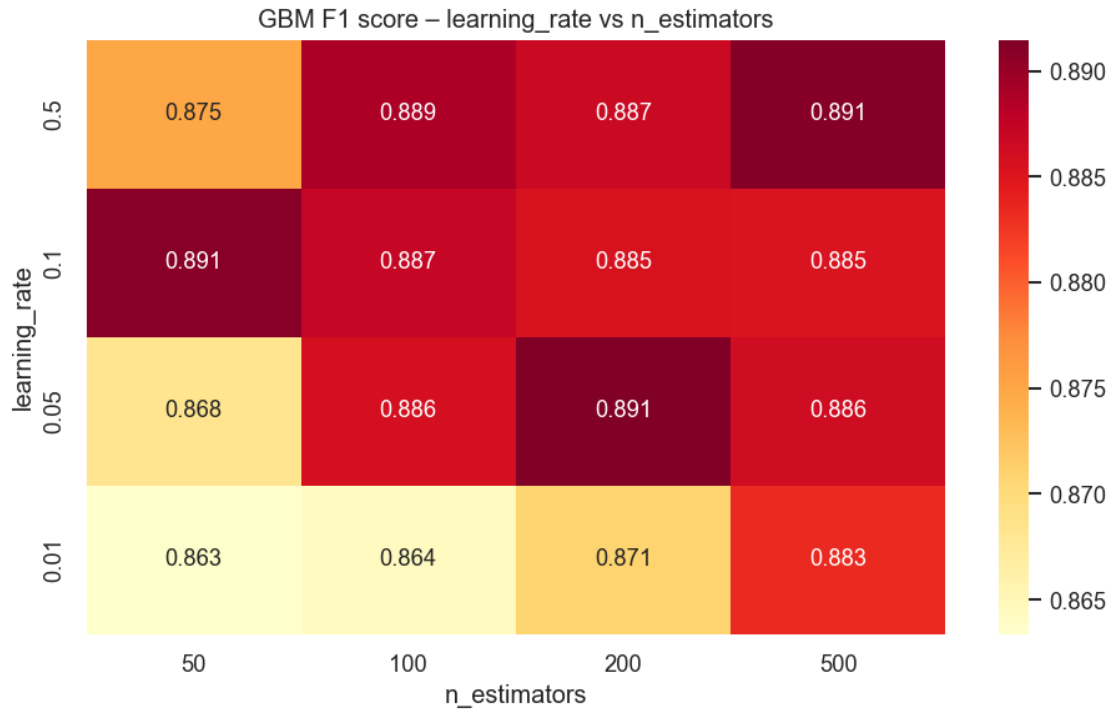
Zadanie: Dla kombinacji `learning_rate` i `n_estimators` oblicz F1 testowe.

```
[22]: # === KROK 2: learning_rate vs n_estimators na zbiorze syntetycznym ===
# Używamy zbioru z Zadania 1
lr_grid = [0.5, 0.1, 0.05, 0.01]
n_est_gb = [50, 100, 200, 500]

heat_data = np.zeros((len(lr_grid), len(n_est_gb)))

for i, lr in enumerate(lr_grid):
    for j, n in enumerate(n_est_gb):
        # TODO: Utwórz GradientBoostingClassifier z max_depth=3,
        #         podanym learning_rate i n_estimators, random_state=RANDOM_STATE
        gb = GradientBoostingClassifier(max_depth=3, learning_rate=lr,
        ↪n_estimators=n, random_state=RANDOM_STATE)
        gb.fit(X_train, y_train)
        heat_data[i, j] = f1_score(y_test, gb.predict(X_test))

fig, ax = plt.subplots(figsize=(8, 5))
sns.heatmap(heat_data, annot=True, fmt='.3f',
            xticklabels=n_est_gb, yticklabels=lr_grid,
            cmap='YlOrRd', ax=ax)
ax.set_xlabel('n_estimators')
ax.set_ylabel('learning_rate')
ax.set_title('GBM F1 score - learning_rate vs n_estimators')
plt.tight_layout()
plt.show()
```

1.4.4 Krok 3: RandomizedSearchCV – automatyczny tuning

Zadanie: Użyj RandomizedSearchCV do znalezienia najlepszych hiperparametrów GBM.

```
[16]: # === KROK 3: RandomizedSearchCV dla GBM ===
param_dist = {
    'n_estimators': [50, 100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1, 0.2, 0.5],
    'max_depth': [2, 3, 4, 5],
    'subsample': [0.6, 0.8, 1.0],
    'max_features': ['sqrt', 0.5, None],
}

# TODO: Utwórz RandomizedSearchCV dla GradientBoostingClassifier
#       n_iter=20, cv=3, scoring='f1', random_state=RANDOM_STATE, n_jobs=-1
rscv = RandomizedSearchCV(
    estimator=GradientBoostingClassifier(random_state=RANDOM_STATE),
    param_distributions=param_dist,
    n_iter=20,
    cv=3,
    scoring='f1',
    random_state=RANDOM_STATE,
    n_jobs=-1,
)
```

```

rscv.fit(X_train, y_train)

print("Najlepsze parametry:", rscv.best_params_)
print(f"Najlepszy F1 (CV): {rscv.best_score_:.4f}")
print(f"F1 na zbiorze testowym: {f1_score(y_test, rscv.predict(X_test)):.4f}")

```

Najlepsze parametry: {'subsample': 0.8, 'n_estimators': 200, 'max_features': 'sqrt', 'max_depth': 5, 'learning_rate': 0.05}
 Najlepszy F1 (CV): 0.9065
 F1 na zbiorze testowym: 0.8816

1.4.5 Krok 4: Finalne porównanie – RF vs AdaBoost vs GBM

Zadanie: Porównaj wszystkie trzy metody na tym samym zbiorze. Użyj najlepszych parametrów z poprzednich kroków.

```

[17]: # === KROK 4: Porównanie RF vs AdaBoost vs GBM ===
try:
    gb_model = GradientBoostingClassifier(**rscv.best_params_,
    ↪random_state=RANDOM_STATE)
except NameError:
    gb_model = GradientBoostingClassifier(
        n_estimators=200, learning_rate=0.05, max_depth=3,
        random_state=RANDOM_STATE)

models = {
    'Random Forest': RandomForestClassifier(
        n_estimators=200, max_features='sqrt',
        oob_score=True, random_state=RANDOM_STATE),

    'AdaBoost': AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=1),
        n_estimators=200, learning_rate=1.0,
        random_state=RANDOM_STATE),

    # TODO: Dodaj GBM z najlepszymi parametrami znalezionymi w Kroku 3
    #       lub użyj n_estimators=200, learning_rate=0.05, max_depth=3
    'Gradient Boosting': gb_model,
}

comparison = []
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    cv_f1 = cross_val_score(model, X, y, cv=5, scoring='f1').mean()
    comparison.append({
        'Model': name,
        'Accuracy': accuracy_score(y_test, pred),
    })

```

```

        'F1 (test)': f1_score(y_test, pred),
        'F1 (CV-5)': cv_f1,
    })

comp_df = pd.DataFrame(comparison).set_index('Model')
print(comp_df.round(4).to_string())

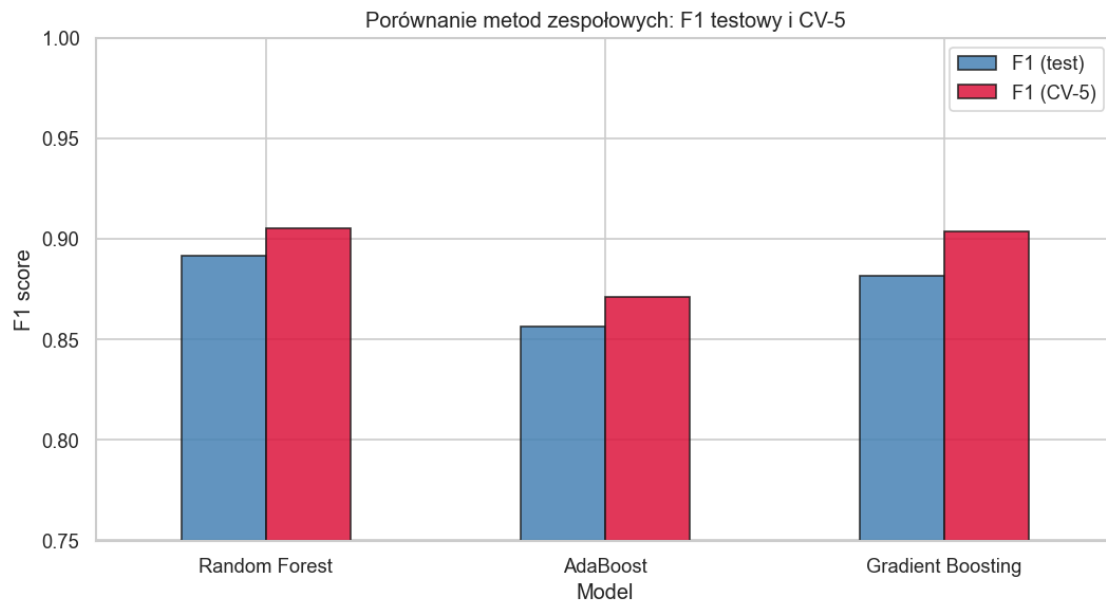
```

	Accuracy	F1 (test)	F1 (CV-5)
Model			
Random Forest	0.888	0.8915	0.9051
AdaBoost	0.852	0.8566	0.8711
Gradient Boosting	0.878	0.8816	0.9035

```

[18]: # Wizualizacja porównania
comp_df[['F1 (test)', 'F1 (CV-5)']].plot(
    kind='bar', figsize=(9, 5), rot=0,
    color=['steelblue', 'crimson'], edgecolor='k', alpha=0.85
)
plt.ylabel('F1 score')
plt.title('Porównanie metod zespołowych: F1 testowy i CV-5')
plt.legend()
plt.ylim(0.75, 1.0)
plt.tight_layout()
plt.show()

```



1.4.6 Pytania do Zadania 3

P3.1 Patrząc na heatmapę (Krok 2): jaki jest efekt zmniejszania `learning_rate` przy stałej liczbie drzew? Co musisz zrobić, aby skompensować mały `learning_rate`?

P3.2 Opisz obserwowane pseudoreszty z Kroku 1 – jak zmieniają się ich wartości w kolejnych iteracjach? Co to mówi o tym, czego uczy się każde kolejne drzewo?

P3.3 W ostatecznym porównaniu (Krok 4): która metoda osiąga najlepszy wynik? Czy GBM zawsze jest lepszy od RF? Jakie są kompromisy?

P3.4 (Pytanie otwarte) Na wykładzie wspomniano o XGBoost, LightGBM i CatBoost. Jaka kluczowa innowacja LightGBM pozwala mu działać szybciej od standardowego GBM przy dużych zbiorach danych?

Tvoja odpowiedź:

P3.1:

Zmniejszanie `learning_rate` osłabia wkład pojedynczego drzewa, więc zwykle potrzebna jest większa liczba estymatorów. Zbyt mały `learning_rate` przy małej liczbie drzew prowadzi do niedouczenia.

P3.2:

Pseudoreszty pokazują błąd pozostały po aktualnym modelu. W kolejnych iteracjach drzewa dopasowują się do coraz mniejszych i bardziej lokalnych reszt, czyli uczą się tego, czego poprzednie drzewa jeszcze nie wyjaśniły.

P3.3:

W finalnym porównaniu najlepszy wynik testowy uzyskał Random Forest: $F1 = 0.8915$. Gradient Boosting był bardzo blisko, z $F1 = 0.8816$ i $CV-5 = 0.9035$, ale nie przebił RF. GBM nie zawsze jest lepszy od RF, ponieważ wymaga dokładniejszego strojenia i jest bardziej wrażliwy na dobór `learning_rate`, liczby drzew oraz głębokości.

P3.4:

LightGBM przyspiesza uczenie m.in. przez histogramowe grupowanie wartości cech oraz strategię wzrostu drzew leaf-wise. W praktyce zmniejsza to koszt obliczeń na dużych zbiorach, a dodatkowe techniki takie jak GOSS i EFB ograniczają liczbę próbek lub cech używanych w kolejnych podziałach.

1.5 Podsumowanie laboratorium

Tabela na podstawie obserwacji z zadań, prezentacji i wiedzy ogólnej:

Metoda	Paradygmat	Co redukowaliśmy	Kluczowa obserwacja
Random Forest	Bagging: bootstrap + losowy podzbiór cech	Głównie wariancję i korelację drzew	OOB dobrze przybliża jakość modelu, a <code>max_features='sqrt'</code> pomaga przez dekorrelację drzew.

Metoda	Paradygmat	Co redukowaliśmy	Kluczowa obserwacja
AdaBoost	Boosting sekwencyjny z ważeniem próbek	Bias przez skupianie się na trudnych przykładach	Metoda może dobrze działać na trudnych danych, ale jest wrażliwa na szum, outliery i błędne etykiety.
Gradient Boosting	Boosting gradientowy dopasowywany do pseudoreszt	Błąd modelu przez minimalizację funkcji straty	Wynik zależy od kompromisu <code>learning_rate</code> i <code>n_estimators</code> ; metoda wymaga strojenia, ale może osiągać bardzo dobre rezultaty.

Literatura (z wykładu): - Breiman (2001), *Machine Learning* – oryginalny artykuł RF - Friedman (2001), *Ann. Stat.* – Greedy Function Approximation (GBM) - Ke et al. (2017), NeurIPS – LightGBM paper - Géron (2022), *Uczenie maszynowe z użyciem Scikit-Learn*, r. 7